



Ontologia Minecraft — Java Edition

1.21.11

Este projeto foi desenvolvido no âmbito da unidade curricular de **Representação e Processamento de Conhecimento na Web (RPCW)** da Universidade do Minho, 2025/2026.

1. Objetivo

O objetivo deste projeto é especificar e implementar uma ontologia para o domínio do **Minecraft Java Edition 1.21.11**, representando elementos relevantes do modo sobrevivência: blocos, items, entidades, mobs, biomas, dimensões, estruturas, receitas, encantamentos, efeitos e progressão de materiais.

A ontologia é explorada através de uma aplicação web em **Flask**, ligada a um repositório **GraphDB**. A aplicação permite navegar por classes e indivíduos, consultar propriedades diretas e inversas, executar queries de competência e aumentar a ABox através da criação de recursos e inserção de conhecimento validado.

2. Fontes de dados

A fonte principal de dados foi o dataset **PrismarineJS/minecraft-data**, versão **1.21.11**, usado para extrair informação sobre blocos, items, entidades, biomas, alimentos, encantamentos, receitas e efeitos.

Como nem toda a informação necessária para as queries de competência se encontrava explícita ou normalizada nesse dataset, foram também criados ficheiros manuais em `data/manual/`. Estes dados complementares cobrem drops de mobs, biomas de spawn, tiers de materiais, requisitos mínimos de mineração, dimensões, estruturas, items necessários para aceder a dimensões, propriedades específicas de mobs e blocos afetados pela gravidade.

A geração final combina a TBox definida manualmente com os indivíduos produzidos automaticamente a partir destes dados.

3. Perguntas de competência

A ontologia e a aplicação foram desenhadas para responder, entre outras, às seguintes perguntas:

1. Que ingredientes são necessários para craftar um item?
2. Que mobs dropam determinado item?
3. Em que biomas pode aparecer determinado mob?
4. Que ferramenta ou tier mínimo é necessário para minerar um bloco?
5. Que items podem receber determinado encantamento?
6. Que estruturas geram num determinado bioma?
7. Qual é a progressão dos tiers de material?
8. Que mobs são imunes ao fogo?
9. Que mobs ardem à luz do sol?
10. Que items são necessários para aceder a uma dimensão?
11. Que blocos são afetados pela gravidade?
12. Que alimentos têm maior saturação?
13. Que encantamentos são incompatíveis entre si?

Estas perguntas são implementadas como queries SPARQL na página **Queries de Competência** da aplicação web.

4. Estrutura da ontologia

A ontologia está organizada em duas partes:

- **TBox**, definida em `ontology/classes.ttl`, com classes, hierarquias, object properties, data properties, restrições e disjunções;
- **ABox**, gerada automaticamente a partir dos datasets e dos ficheiros manuais.

A ontologia final é produzida em `ontology/minecraft.ttl`, juntando a TBox com os ficheiros RDF gerados pelos exporters.

As principais classes de topo são:

Classe	Papel
GameObject	objetos do jogo, como blocos, itens e entidades
Environment	contextos espaciais, como dimensões, biomas e estruturas
Recipe	processos de produção ou transformação de itens
Enchantment	modificadores aplicáveis a itens
Effect	estados temporários aplicáveis a entidades
MaterialTier	níveis de progressão e qualidade dos materiais

A hierarquia detalhada inclui, por exemplo, `Block`, `Item`, `Entity`, `Mob`, `Biome`, `Structure`, `CraftingRecipe`, `ShapedRecipe`, `ShapelessRecipe`, `WeaponEnchantment`, `ArmorEnchantment`, `BeneficialEffect` e `HarmfulEffect`.

Também foram definidas classes conceptuais ou inferíveis, como `PlaceableBlock`, `LightEmittingBlock`, `TransparentBlock`, `IndestructibleBlock`, `StackableItem`, `NonStackableItem`, `RenewableResource` e `NonRenewableResource`. Algumas destas classes podem não ter instâncias diretas no dataset final, mas foram mantidas porque fazem parte da modelação do domínio e permitem representar conceitos úteis para inferência, validação e extensão futura.

5. Decisões de modelação

Uma decisão importante foi não declarar `Block` e `Item` como classes disjuntas. No Minecraft, vários recursos têm dupla natureza: podem existir como bloco no mundo e como item no inventário. Exemplos como `stone`, `oak_planks` ou `diamond_ore` justificam esta multi-classificação. A classe `PlaceableBlock` representa precisamente a interseção conceptual entre blocos e itens.

As subclasses de `Enchantment` também não foram todas consideradas disjuntas. Alguns encantamentos aplicam-se a vários tipos de equipamento, como `unbreaking` ou `mending`. Por isso, a aplicabilidade foi modelada através de relações como `applicableTo` e `canBeEnchantedWith`.

As receitas foram modeladas como indivíduos de `Recipe`, com especialização em

`ShapedRecipe` e `ShapelessRecipe`. Para receitas shaped, foram usados indivíduos auxiliares `RecipeSlot`, permitindo preservar a posição dos ingredientes através de `slotRow`, `slotColumn` e `slotItem`. Isto permite consultar tanto os ingredientes usados como a estrutura da receita.

Os tiers de material foram representados como indivíduos de `MaterialTier`, por exemplo `Wood`, `Stone`, `Iron`, `Diamond` e `Netherite`. A propriedade `tierOrder` permite ordenar estes tiers e responder a perguntas sobre progressão.

Foram ainda definidas disjunções pontuais quando a separação era segura no domínio, como `HostileMob` disjoint with `PassiveMob`, `BeneficialEffect` disjoint with `HarmfulEffect` e `ShapedRecipe` disjoint with `ShapelessRecipe`.

6. Propriedades principais

A ontologia define object properties para representar relações entre indivíduos. As mais relevantes são:

- `drops` / `droppedBy`, para relacionar mobs com itens dropados;
- `spawnsIn` / `hasSpawn`, para relacionar mobs com biomas;
- `minedWith`, `canMine` e `requiresMinTier`, para requisitos de mineração;
- `generatesIn` / `hasStructure`, para estruturas geradas em biomas;
- `applicableTo` / `canBeEnchantedWith`, para encantamentos aplicáveis a itens;
- `incompatibleWith`, para incompatibilidades entre encantamentos;
- `hasIngredient`, `usedIn`, `produces` e `craftedBy`, para receitas;
- `requiredToEnter` / `opensAccess`, para acesso a dimensões;
- `hasSlot` e `slotItem`, para estrutura interna de receitas shaped.

As data properties representam atributos literais. Foram usadas propriedades específicas por tipo de recurso, como `blockID`, `itemID`, `entityID`, `biomeID`, `effectID` e `enchantmentID`, evitando ambiguidades em indivíduos que podem ser classificados em mais do que uma classe. Outros exemplos incluem `stackSize`, `hardness`, `blastResistance`, `emitLight`, `foodPoints`, `saturation`, `health`, `experienceDrop`, `maxLevel`, `temperature`, `outputQuantity` e `tierOrder`.

7. Geração da ontologia

A geração é feita automaticamente através do comando:

```
python scripts/build_ontology.py
```

Este script executa os exporters, gera os ficheiros `scripts/ontology/data_*.ttl`, junta-os com `ontology/classes.ttl`, produz `ontology/minecraft.ttl` e valida se o ficheiro Turtle final pode ser carregado.

Na última geração foram obtidas as seguintes estatísticas:

```
Triples: 106319
Classes: 164
Object properties: 43
Data properties: 59
```

Os ficheiros `data_*.ttl` são resultados gerados automaticamente e não devem ser editados manualmente. Alterações permanentes ao conteúdo devem ser feitas nos datasets de origem, nos ficheiros manuais ou na TBox, seguidas de nova geração.

8. Aplicação web

A aplicação web foi desenvolvida em Flask e comunica com o GraphDB através do endpoint SPARQL. Permite consultar a ontologia de forma navegável, sem obrigar o utilizador a escrever SPARQL manualmente.

Funcionalidades principais:

- página inicial com descrição do projeto;
- listagem de classes;
- consulta dos indivíduos de cada classe;
- página de detalhe de cada recurso;
- visualização de propriedades diretas e relações inversas;
- execução de queries de competência;
- execução de SPARQL livre;

- criação de novos recursos na ABox;
- inserção guiada de conhecimento;
- inserção de triples genéricas com validação.

A aplicação responde assim aos dois requisitos práticos do projeto: explorar a ontologia e permitir o seu aumento a partir da interface web.

9. Extensão da ontologia pela aplicação

A extensão da ontologia é feita ao nível da ABox, mantendo a TBox controlada nos ficheiros Turtle.

Existem três modos principais:

1. **Adicionar Recurso**, para criar novos indivíduos, indicando nome local, classe, label e descrição opcional.
2. **Adicionar Conhecimento**, para inserir padrões frequentes, como drops de mobs, spawns em biomas, estruturas em biomas, requisitos de mineração, acesso a dimensões ou propriedades de mobs.
3. **Adicionar Relação**, para inserir triples genéricas, validando o predicado, o tipo esperado do objeto, o domínio e o range quando essa informação existe na TBox.

Exemplo de triple válida:

```
creeper spawnsIn plains
```

Exemplo de triple rejeitada:

```
creeper spawnsIn gunpowder
```

A segunda é rejeitada porque `spawnsIn` espera um recurso da classe `Biome`, enquanto `gunpowder` é um `Item`.

As alterações feitas pela aplicação são inseridas no repositório GraphDB em tempo de execução. Para as tornar permanentes no ficheiro base, devem ser exportadas do GraphDB ou adicionadas aos ficheiros em `data/manual/` e depois regeneradas com

```
python scripts/build_ontology.py .
```

10. Execução

Para gerar a ontologia:

```
python scripts/build_ontology.py
```

No GraphDB deve ser criado um repositório chamado `minecraft` e importado o ficheiro:

```
ontology/minecraft.ttl
```

O endpoint esperado é:

```
http://localhost:7200/repositories/minecraft
```

Para correr a aplicação:

```
cd webapp  
pip install -r requirements.txt  
python app.py
```

Depois, abrir no browser:

```
http://127.0.0.1:5000
```

Exemplos úteis para demonstração:

Funcionalidade	Input
Ingredientes de receita	torch
Mobs que dropam item	gunpowder
Biomassas de spawn de mob	creeper

Funcionalidade	Input
Ferramenta para minerar bloco	diamond_ore
Items aplicáveis a encantamento	fortune
Estruturas por bioma	desert
Items necessários para dimensão	nether

11. Estrutura do projeto

A organização principal da pasta `Projeto2026` é:

```
data/      datasets originais e dados manuais
ontology/  TBox e ontologia final
scripts/   exporters e geração da ABox
webapp/    aplicação Flask
README.md  relatório do projeto
```

12. Limitações

Alguma informação teve de ser adicionada manualmente porque não se encontrava totalmente disponível ou normalizada no dataset PrismarineJS. Os dados manuais cobrem um subconjunto representativo do domínio, suficiente para demonstrar as queries de competência e a extensibilidade da ontologia.

A cobertura pode ser aumentada adicionando novas entradas aos ficheiros em `data/manual/` e regenerando a ontologia. A aplicação permite inserir conhecimento diretamente no GraphDB, mas essas alterações só se tornam permanentes no ficheiro final se forem exportadas ou integradas nos dados de origem.